

Bi-Weekly Status Report 4

Project Name: CodeTrail – Real-Time Collaborative Code Editor with Contribution Tracking

Course: WS & SOA (IT645) | Dhirubhai Ambani University (Formerly DA-IICT)

Report Period: 14/09/2026 – 27/09/2026 (Weeks 8–9)

Date Submitted: 30/09/2026

Prepared By: Shaikh Maryam on behalf of CodeTrail Team

Project Status Summary¹

Sprint / Milestone	SCRUM Sprint 4 (Bi-Weekly 4) – Module 4 Delivery: Code Execution & Module 5: Contribution Logging			
% Complete (overall)	62.5% (5 of 8 functional modules delivered On Track)			
Indicator	Current	Previous	Trend	Status
Overall Project	62.5%	37.5%	↑	On Track
Schedule	On Schedule	On Schedule	→	On Track
Scope	5/8 Modules	3/8 Modules	↑	On Track
Quality	Piston API and SHA-256 verification stable	Stable	→	On Track
Team Contribution	8/8 members contributed	8/8 members contributed	→	On Track
Dependencies	Piston API fully integrated	Piston API 60%	↑	On Track

Summary Notes:

The CodeTrail team successfully completed and merged Module 4 (Code Execution Engine) and Module 5 (Contribution Dossier Logging) in Bi-Weekly 4 (14/09/2026–27/09/2026, Weeks 8–9). Key deliverables included a complete Piston API integration for multiple language runtimes, the core SHA-256 cryptographic proof-of-work pipeline for tamper-evident activity logs, and a full telemetry workspace panel. Furthermore, the CI/CD deployment pipeline was finalized and hosted on Vercel.

Key Accomplishments in Two Weeks

- Module 4 Delivery – Code Execution Engine:** Implemented a full code execution controller with a local Piston server, allowing real-time sandboxed code execution in languages like JavaScript, Python, C++, Java, and Go. Real-time stdout/stderr mapping and custom stdin handling were fully integrated into the ModularWorkspace execution runner. (Commits 3a9d94ca, 4edb37c0, 32a2407e, d700a61d)

¹For details on indicators and status levels, see last page.

- **SHA-256 Proof-of-Work Activity Logging (CT-460):** Developed the cryptographic activity logging pipeline that records tamper-evident contribution history. Implemented a verification engine that secures logs with proof-of-work, alongside auto-recording features embedded into the workspace controller. (Commits `c035413e`, `b65c6b20`)
- **Module 5 Delivery – Contribution Dossier Telemetry:** Launched the workspace telemetry panel that visualizes the recorded activity logs, tracks participant contribution metrics, and generates exportable audit reports. The telemetry layer interfaces via Socket.IO to keep metrics synchronized in real-time. (Commits `55333ac2`, `c3cae57e`, `19819efb`)
- **Activity Validation & Tamper Simulation:** Built visual verification indicators to track the integrity of cryptographic logs, including tamper simulation features for robust QA testing. (Commit `b97fc9d1`)
- **CI/CD Deployment Setup:** Initialized and deployed the frontend via Vercel, adding specific Vite build configurations and post-install hooks to ensure zero-downtime deployment for future CI iterations. (Commits `c66702cf`, `a02b7690`)

Planned vs. Actual

Task/ Deliverable	Planned Completion	Current Forecast	Actual Status	Notes
Module 3: Col- laborative Edi- tor	Sep 14	—	Done	Delivered in Bi- Weekly 3
Module 4: Code Execu- tion	Sep 28	—	Done	Piston API fully in- tegrated
SHA-256 Con- tribution Log- ging	Sep 28	—	Done	Completed via CT- 460 pipeline
Module 5: Con- tribution Dossier	Oct 12	—	Done	Delivered ahead of schedule
Vercel CI/CD Setup	Oct 12	—	Done	Successfully de- ployed

Project Risks, Issues, & Dependencies

Type (ID)	Description	Impact	Owner	Mitigation / Next Step
Dependency (D-02)	Third-party Piston API execution rate limits and latency variability	Medium	Meet Sheth / Dudhat Monar	Resolved: Integrated timeout guards and rate limit handling via local server fallbacks.
Issue (I-03)	Vite build failures during Vercel deployment due to conflict markers	High	Mahi Kansara	Resolved: Fixed conflict markers in ModularWorkspace and implemented rigorous CI checks.
Risk (R-05)	Overhead of real-time SHA-256 hashing during rapid socket code updates	Medium	Thakkar Manushri	Debounced payload logging to limit hash calculations per minute per user.

Project Decisions

Decision	Rationale	Impact	Decided By	Date
Local Piston Server caching	Mitigate third-party rate limits and latency by caching specific execution runtimes.	Stable code execution across various languages with minimized delay.	Team	Sep. 19
Client-side Telemetry Sync (Socket)	Allows immediate visualization of contribution data without excessive database polling.	Reduced database load and instant UX feedback on contribution dossier.	Mahi	Sep. 28
Vercel Build Configuration	Ensures proper Vite compilation by resolving environment and dependency specific conflicts on deployment.	Enables automated continuous deployment.	Mahi	Sep. 28

Project Scope Changes

Upcoming Work (Next Two Weeks' Plan – Bi-Weekly 5 Target)

- System integration testing and performance optimizations on the completed modules.
- Expand Postman API test suite to cover all telemetry endpoints.

Change	Action	Reason	Schedule Impact	Approved By
None	—	Scope remains fully aligned with project proposal and approved SDD specifications.	None	—

- Enhance UI layout responsiveness on mobile views for the telemetry dossier.
- Start working towards final presentation preparation and documentation compilation.

Team Member Contributions

Name: Meet Sheth (202512057)

Role: Backend Architect / System Designer

Tasks Completed	Engineered the core Code Execution Controller and architected the local Piston execution server to process real-time code execution requests. Seamlessly integrated execution endpoints into the modular workspace backend, ensuring robust isolation and multi-language runtime stability (Commit 4edb37c0).
Contribution Type	Development / Architecture
Hours / Capacity	30/30 hrs logged
Evidence	<code>backend/controllers/executionController.js</code>

Name: Rishika Shah (202512041)

Role: Frontend Developer / UI Engineer

Tasks Completed	Resolved critical UI rendering anomalies within the Code Execution module and engineered a robust custom standard input (stdin) handling mechanism. Enhanced the overall execution engine interface to provide real-time interactive terminal feedback for end-users (Commit 32a2407e).
Contribution Type	Development / Testing
Hours / Capacity	30/30 hrs logged
Evidence	<code>frontend/src/modules/workspace/ModularWorkspace.jsx</code>

Name: Mahi Kansara (202512078)

Role: Frontend Developer / Socket.IO Integration Lead

Tasks Completed	Led the development of real-time telemetry (CT-164/167), implementing high-precision timestamp recording and robust Socket.IO synchronization. Architected the Vercel CI/CD pipeline, resolving complex Vite build dependencies to establish zero-downtime automated deployments (Commits 19819efb , c66702cf , a02b7690).
Contribution Type	Development / DevOps
Hours / Capacity	30/30 hrs logged
Evidence	<code>vercel.json</code> , <code>package.json</code> , Telemetry socket integrations

Name: Thakkar Manushri (202512122)

Role: Project Manager / DevOps / Backend Developer

Tasks Completed	Architected and implemented the core SHA-256 Proof-of-Work cryptographic logging pipeline to establish tamper-evident activity records. Securely integrated the proof-of-work auto-recording mechanism into the workspace controller, bridging backend security with real-time audit trails (Commits c035413e, b65c6b20).
Contribution Type	Development / Architecture
Hours / Capacity	30/30 hrs logged
Evidence	backend/utils/cryptoProof.js, backend/controllers/workspaceController

Name: Anistina Dsouza (202512081)

Role: Full-Stack Developer / Module Lead

Tasks Completed	Advanced the execution sandbox capabilities by developing an editable interactive runner, comprehensive diagnostic overlays, and real-time execution status badges. Formulated and executed rigorous backend validation suites for the Piston execution engine to guarantee runtime reliability (Commits 635432d8, d700a61d).
Contribution Type	Development / Testing
Hours / Capacity	30/30 hrs logged
Evidence	backend/tests/index.js, backend/scripts/simulateTamperValidation.js

Name: Ruchita Patadiya (202512070)

Role: Frontend Developer / Backend Developer

Tasks Completed	Engineered the client-side Contribution Dossier and workspace telemetry visualization panels. Implemented comprehensive contribution tracking metrics and developed the core functionality to compile and export verifiable audit reports in both markdown and JSON formats (Commit 55333ac2).
Contribution Type	Development
Hours / Capacity	30/30 hrs logged
Evidence	frontend/src/modules/telemetry/ContributionDossier.jsx, frontend/src/utils/auditExport.js

Name: Shaikh Maryam (202512074)

Role: Full-Stack Developer / UI Architect

Tasks Completed	Led the frontend UI architecture for the Code Execution Engine, optimizing responsive layouts and language support. Spearheaded complex Git merge conflict resolutions to seamlessly integrate the Telemetry & Audit Proof-of-Work modules (CT-164/167) into the core workspace environment. Systematically refactored the ModularWorkspace component to incorporate the new Telemetry Drawer while ensuring cross-component UI consistency (Commits 3a9d94ca, a601728f, 4cfc152b).
Contribution Type	Development / Project Management
Hours / Capacity	30/30 hrs logged
Evidence	frontend/src/modules/workspace/ModularWorkspace.jsx

Name: Dudhat Monar (202512019)

Role: Backend Developer / QA Engineer

Tasks Completed	Spearheaded the Quality Assurance (QA) and security validation of the cryptographic pipeline. Developed real-time visual verification indicators, automated activity log validation scripts, and a tamper simulation suite to rigorously test the integrity of the Proof-of-Work telemetry data (Commit <code>b97fc9d1</code>).
Contribution Type	Development / Testing
Hours / Capacity	30/30 hrs logged
Evidence	<code>backend/scripts/simulateTamperValidation.js</code> , <code>frontend/src/modules/telemetry/ContributionDossier.jsx</code>

Guidelines (for reference, not to be submitted in your status report):

- Section 1 summarizes the overall project status.
- Sections 2–5 capture team-wide progress.
- Section 6 is the per-person breakdown – keep individual entries short (3–5 bullet points max) so the report does not balloon as your project progresses / team grows.
- The status indicators at the top give stakeholders a one-glance read before they dig into details – useful if this report goes to non-technical stakeholders too.
- Six project status indicators have the following context:
 - Overall Progress — % of project scope completed
 - Schedule — % of planned work completed
 - Scope — % of approved requirements completed
 - Quality — test pass rate / critical defects
 - Team Contribution — % of assigned deliverables completed
 - Dependencies — % of critical dependencies resolved
- Status levels:
 - On Track — expected to meet current milestone
 - Needs Attention — issue exists but recovery is likely
 - At Risk — milestone/deliverable may be missed without intervention
 - Blocked — progress cannot continue without resolving a dependency
- Contribution Type with examples (Add more types, if needed)
 - Development — Code, APIs, UI, database
 - Architecture — Architecture/design decisions
 - Testing — Test cases, automation, defect verification
 - Documentation — SRS, design docs, reports
 - DevOps — CI/CD, deployment, containers
 - Research — Literature review, technical investigation
 - Project Management — Planning, coordination
 - Presentation — Demo, presentation, client communication